

Software Architecture Document — Security Lab

Версия: 1.0

Проект: Security Lab — система сканирования безопасности Kubernetes и контейнерных образов

Статус: актуально

1. Введение

1.1. Назначение документа

Документ описывает архитектуру системы Security Lab: её структуру, ключевые компоненты, потоки данных, развёртывание и принятые архитектурные решения. Предназначен для разработчиков, сопровождающих систему, и проверяющих.

1.2. Область применения

Security Lab — лабораторно-учебная платформа для оценки защищённости Kubernetes-кластера и образов контейнеров. Система разворачивает кластер, устанавливает набор сканеров безопасности, запускает сканирования, агрегирует их результаты и предоставляет веб-интерфейс для анализа.

1.3. Термины и сокращения

Термин	Значение
CVE	Common Vulnerabilities and Exposures — идентификатор уязвимости
CVSS	Common Vulnerability Scoring System — оценка критичности (0–10)
SBOM	Software Bill of Materials — состав ПО (список компонентов)
Misconfiguration	Небезопасная конфигурация k8s-ресурса
CIS Benchmark	Набор best-practice проверок безопасности кластера
NVD	National Vulnerability Database — база уязвимостей NIST
Trivy / Gype / Syft / kube-bench	Сканеры/инструменты безопасности

2. Архитектурные цели и ограничения

2.1. Цели

- **Единая точка** для запуска разнотипных сканирований (образы, ресурсы кластера, политики CIS).
- **Агрегация** результатов разных сканеров в общую модель находок.
- **Неблокирующий** запуск сканирований (они длятся минуты).
- **Обогащение** находок внешними данными (NVD) по требованию.
- **Наглядное** представление результатов с фильтрацией по категориям и важности.

2.2. Ограничения и допущения

- Хранилище — **SQLite** (один файл), достаточно для лабораторного масштаба.
- Бэкенд работает **на мастер-узле** кластера: имеет локальный доступ к kubectl, helm и установленным сканерам.
- Сканеры — внешние CLI-инструменты, вызываются через subprocess.
- Обращения к NVD ограничены rate-limit'ом (выше с API-ключом).
- Аутентификация — простой статический API-key (для лабораторного контура).

3. Представление вариантов использования (Use Cases)

Ключевые сценарии, формирующие архитектуру:

#	Сценарий	Действующее лицо
UC1	Запустить скан образов из helm-чарта	Инженер безопасности
UC2	Просканировать namespace на CVE и мисконфиги	Инженер безопасности
UC3	Просканировать весь кластер (Trivy + kube-bench/CIS)	Инженер безопасности
UC4	Просмотреть находки скана с фильтрами	Инженер безопасности
UC5	Обогатить CVE данными из NVD	Инженер безопасности
UC6	Сравнить два скана (что починили / что появилось)	Инженер безопасности
UC7	Скачать SBOM образа	Инженер безопасности / аудитор



4. Логическое представление (Logical View)

4.1. Слои системы

Presentation (React + TypeScript) features/scans · features/vulnerabilities · features/sbom api/scan.ts (REST-клиент)
API (FastAPI, server.py) эндпоинты сканов · уязвимостей · SBOM · namespaces · stats verify_api_key (auth)
Domain / Services (server.py) запуск сканеров · парсеры вывода · merge-логика CVE фоновые задачи (BackgroundTasks)
Integration subprocess → trivy / gype / syft / kube-bench / kubect1 HTTP → NVD API
Persistence (SQLite) scans · scan_images · scan_findings · vulnerabilities · scan_sbom

4.2. Ключевые модули бэкенда

Модуль (функции)	Ответственность
init_db, get_conn	Инициализация схемы, доступ к БД, миграции
run_cmd	Унифицированный запуск CLI с таймаутом
run_trivy_image, parse_trivy_findings	Скан образа Trivy и парсинг
run_gype_image, parse_gype_findings	Скан образа Gype (через Syft SBOM)
parse_trivy_k8s_report	Парсинг trivy k8s (vuln + misconfig)
_parse_kube_bench_json	Парсинг kube-bench (CIS)
run*_background	Фоновые задачи сканирования + смена статуса
upsert_vulnerability	Каталог CVE: дополнение без перезатирания
sync_nvd_for_scan, fetch_nvd_cve, parse_nvd_item	Обогащение из NVD
save_finding, save_sbom, save_images	Запись результатов

4.3. Модули фронтенда

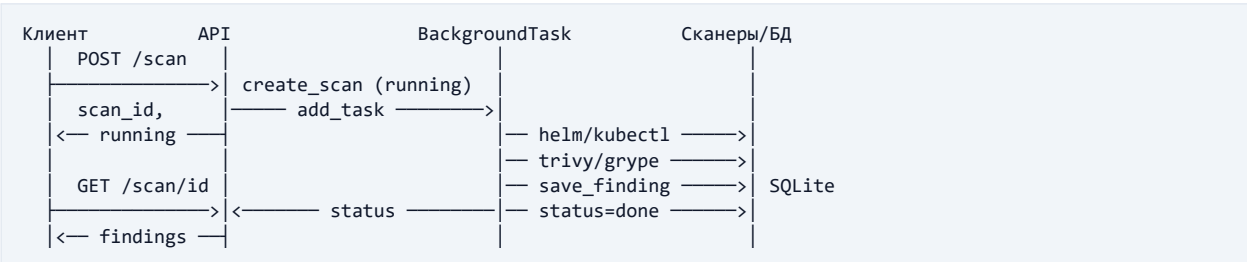
Модуль	Ответственность
api/scan.ts	REST-клиент, прокидывание API-key
features/scans/ScanList	Список сканов образов (категория image)
features/scans/ScanDetailsPage	Детали скана образов, фильтры
features/scans/cluster/K8sScanPage	Запуск+результаты namespace-сканов
features/scans/cluster/ClusterScanPage	Запуск+результаты кластер/политик
features/scans/ScanFindingsPanel	Переиспользуемая панель находок
useScanFindings, useScansOfType	Хуки загрузки с авто-опросом статуса
features/vulnerabilities	Каталог CVE

Модуль	Ответственность
features/sbom	Просмотр/выгрузка SBOM

5. Представление процессов (Process View)

5.1. Асинхронное сканирование

Сканы длятся минуты, поэтому запуск **неблокирующий**: HTTP-запрос сразу возвращает `scan_id` и `status=running`, а работа идёт в `BackgroundTasks`. Фронтенд опрашивает статус.



Опрос на фронте реализован хуками `useScanFindings / useScansOfType`: пока статус `running/created` — повторный запрос каждые 4–5 секунд.

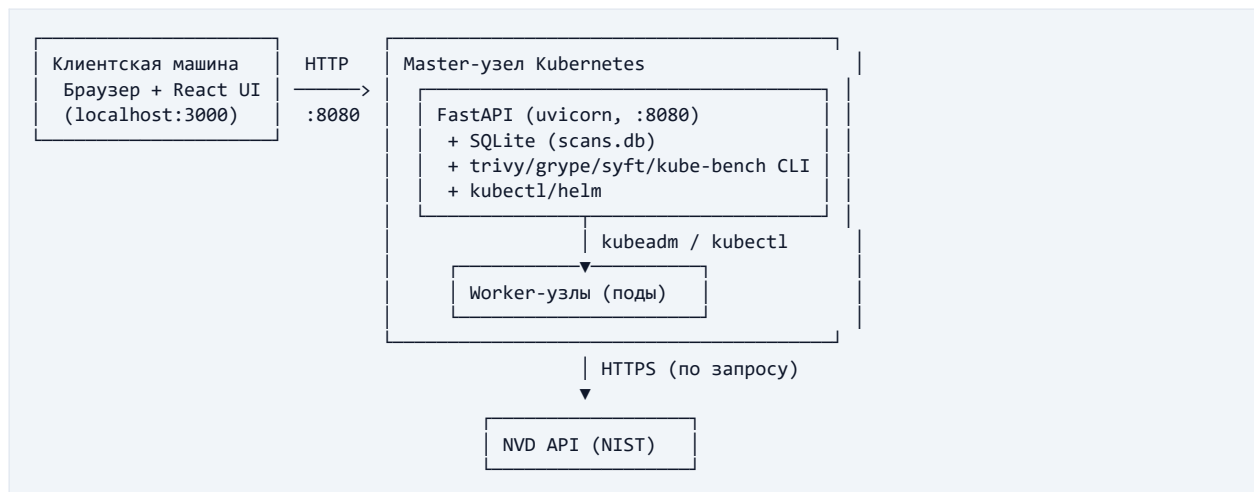
5.2. Обогащение NVD

Запускается **только вручную** (UC5). Батч по скану идёт в фоне; между запросами выдерживается пауза согласно `rate-limit NVD` (0.7с с ключом, 6с без). Merge-логика **дополняет** каталог CVE, не перезаписывая уже известные значения.

5.3. Очистка артефактов

`trivy k8s` создаёт в кластере `job node-collector`. Перед каждым кластерным сканом выполняется очистка остатков (`_cleanup_node_collector`), чтобы избежать ошибки «`already exists`».

6. Представление развёртывания (Deployment View)



Развёртывание кластера и установка сканеров автоматизированы Ansible-плейбуками 00–07 (bootstrap → containerd → master → network → workers → trivy → grype → kube-bench).

7. Представление данных (Data View)

Хранилище — SQLite (server/data/scans.db).

```
scans (id PK, ts, namespace, release, status)
├─< scan_images (scan_id, image)
├─< scan_findings (scan_id, scanner, severity, target, title,
                  cve_id, pkg_name, installed_version, fixed_version,
                  cvss_score, description, references_json, finding_type)
└─< scan_sbom (scan_id, image, format, content, created_at)

vulnerabilities (cve_id PK, severity, cvss_score, title, description,
                 published_at, modified_at, source, references_json,
                 raw_json, updated_at) ← связь по cve_id (LEFT JOIN)
```

- Тип скана определяется по release/namespace (см. категории в README).
- scan_findings.cve_id связывается с каталогом vulnerabilities через LEFT JOIN при выдаче деталей, чтобы подтянуть обогащённые данные (CVSS, описание).
- finding_type различает уязвимости (vuln) и мисконфиги (misconfig).

8. Размер и производительность

- Сканы выполняются асинхронно — API остаётся отзывчивым.
- Таймауты на CLI: образ — 300с, кластер — 600с, kubectl/helm — 30–120с.
- Пагинация выдачи находок (до 200 на страницу).
- Индексы: scan_findings(scan_id, severity), scan_findings(cve_id), vulnerabilities(severity), scan_sbom(scan_id).
- Ограничения масштаба: SQLite и однопроцессный фон рассчитаны на лабораторную нагрузку, не на параллельные тяжёлые сканы в проде.

9. Качество и кросс-функциональные требования

9.1. Безопасность

- Аутентификация по заголовку X-API-Key (включается переменной API_KEY).
- Ключ NVD читается из окружения (NVD_API_KEY), не хранится в коде.
- CORS открыт (лабораторный контур; для прода требует ужесточения).

9.2. Надёжность

- Любой сбой фоновой задачи переводит скан в статус failed (видно в UI).
- CLI-вызовы обёрнуты таймаутом — зависший инструмент не вешает систему.
- Очистка остаточных job'ов перед кластерным сканом.

9.3. Сопровождаемость

- Бэкенд — единый модуль с разделением на секции (DB / util / scanners / API).
- Фронтенд — feature-ориентированная структура, переиспользуемые хуки и компоненты.
- Миграции схемы выполняются при старте (init_db).

9.4. Расширяемость

- Новый сканер добавляется как пара «runner + parser» + значение поля scanner.
- Новая категория сканов — через поле release/namespace и фильтр в /scan/list.

10. Архитектурные решения (кратко)

Решение	Обоснование
FastAPI + BackgroundTasks	Простой асинхронный запуск долгих сканов без внешнего брокера
SQLite	Достаточно для лабораторного масштаба, ноль инфраструктуры
Вызов сканеров через CLI	Переиспользование зрелых инструментов вместо своих движков
Trivy + Gpуре параллельно	Перекрёстная валидация находок по образам
NVD по запросу, merge без перезатирания	Контроль над внешними обращениями; сохранность данных
Бэкенд на мастер-узле	Локальный доступ к kubectl/helm/сканерам без проксирования

11. Известные риски и ограничения

- **SQLite** при росте нагрузки/конкуренции станет узким местом → миграция на PostgreSQL.

- **Один процесс** фоновых задач не масштабируется горизонтально → очередь (Celery/RQ) при необходимости.
- **CORS** * и статический API-key недостаточны для продакшна → OAuth/JWT, ограничение origin.
- **Зависимость от версий CLI** сканеров (флаги меняются между релизами).